

THE EVOLUTIONARY CODE LABORATORY

WAFt

Field Manual & Documentation

*“Don’t just build agents. **Breed** them.”*

Version: 1.0.0
License: MIT
Python: 3.10+

Date: January 2026
Author: ctavolazzi
Status: Active Development

A Python framework for directed evolution of self-modifying AI agents

Table of Contents

Introduction 3

 What is WAFT? 3

 Core Pillars 3

Quick Start Guide 4

 Installation 4

 Create Your First Laboratory 4

 The Evolutionary Cycle 4

Core Concepts 5

 The Substrate: Code as DNA 5

 The Physics: Scint System 5

 The Flight Recorder 6

Commands Reference 7

 Core Commands 7

 Empirica Commands 7

 Gamification Commands 8

Project Structure 9

 Key Directories 9

Documentation System 10

 Document Templates 10

 Self-Observation Systems 10

The Scientific Mission 11

 Data Collection 11

 Philosophy 11

Appendix: Quick Reference 12

 Essential Commands 12

 Scint Types Quick Reference 12

 Resources 12

Introduction

The Promise

Waft is a scientific instrument for studying the **physics of artificial cognition** through directed evolution. We don't just create AI agents—we breed them, test them in the crucible of reality, and observe them evolve over thousands of generations.

What is WAFT?

WAFT is an evolutionary code laboratory where:

- **Code is DNA** — Agents can modify their own source code
- **Fitness determines survival** — The Scint system tests agents
- **Everything is recorded** — The Flight Recorder tracks lineage

DNA ⇒ Mutation ⇒ Fitness Test ⇒ Selection → Evolution

Core Pillars

1. The Substrate

Agents write their own Python source code (DNA)

2. The Physics

Scint System tests fitness and kills weak mutations

3. Flight Recorder

Telemetry for phylogenetic tree generation

The ultimate goal: observe a “God-Head” agent emerge from evolution.

Quick Start Guide

Installation

Prerequisites

- Python 3.10+
- uv package manager
- just task runner (optional)

Install WAFT

```
1 # Using uv (recommended)
2 uv tool install waft
3
4 # Or from source
5 git clone https://github.com/ctavolazzi/waft.git
6 cd waft
7 uv sync
8 uv tool install --editable .
```

Create Your First Laboratory

```
1 # Create a new evolutionary laboratory
2 waft new my_laboratory
3
4 # Navigate and verify
5 cd my_laboratory
6 waft verify
```

Your laboratory is ready! You now have the basic structure for evolving agents.

The Evolutionary Cycle

```
1 # 1. Spawn variants with mutations
2 waft spawn --agent RefactorAgent --mutation "improved_prompt.json"
3
4 # 2. Evaluate fitness in the Gym
5 waft eval --agent RefactorAgent
6
7 # 3. Evolve into the fittest variant
8 waft evolve --agent RefactorAgent --generation 5
```

Use
uv
for
best
experier

Core Concepts

The Substrate: Code as DNA

In WAFT, code is DNA. Agents can:

- **Spawn** — Create variants with mutations (code changes)
- **Evolve** — Hot-swap their own code/config
- **Reproduce** — Create children with genetic modifications

Every agent has a unique genome ID (SHA-256 hash)

Genome Structure

```
1 @dataclass
2 class AgentGenome:
3     """The genetic material of an agent."""
4     code: str          # Python source code
5     config: dict       # Configuration parameters
6     prompts: list[str] # System prompts
7
8     @property
9     def genome_id(self) -> str:
10         """SHA-256 hash of the genome."""
11         content = f"{self.code}{json.dumps(self.config)}"
12         return hashlib.sha256(content.encode()).hexdigest()
```

The Physics: Scint System

The **Reality Fracture Detection System** (Scint Gym) serves as the predator that kills weak mutations.

Scint Types (Ontological Errors)	
SYNTAX_TEAR	Formatting errors (JSON, XML, Code)
LOGIC_FRACTURE	Math errors, contradictions
SAFETY_VOID	Harmful content, PII leaks
HALLUCINATION	Fabricated facts, wrong citations

Fitness Calculation

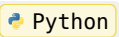
Fitness is measured by three weighted scores:

Stability	Efficiency	Safety
40% weight	30% weight	30% weight
Scint correction	Agent call efficiency	Compliance score

The Flight Recorder

Every evolutionary action is recorded with complete context:

```
1 @dataclass
2 class FlightRecord:
3     genome_id: str      # SHA-256 hash
4     parent_id: str      # Lineage tracking
5     generation: int     # 0 = Genesis
6     event_type: str     # SPAWN, MUTATE, GYM_EVAL, DEATH
7     payload: dict       # Complete context
8     fitness: float      # Gym evaluation score
9     timestamp: datetime
```



This enables:

- Phylogenetic analysis of evolutionary relationships
- Mutation impact measurement
- Fitness landscape mapping
- Dead end detection

Fitness
< 0.5 =
DEATH
(evolutionary
dead
end)

Commands Reference

Core Commands

waft new <name>

Creates a new evolutionary laboratory.

```
1 waft new my_laboratory
2 waft new my_laboratory --path /path/to/target
```

Shell

Options:

- --path, -p: Target directory (default: current)

waft verify

Verifies the project structure.

```
1 waft verify
2 waft verify --path /path/to/project
```

Shell

waft evolve

Run the evolutionary cycle (Spawn → Gym → Select).

```
1 waft evolve --agent RefactorAgent
2 waft evolve --agent RefactorAgent --generations 10
```

Shell

Empirica Commands

Session Management

```
1 waft session create
2 waft session bootstrap
3 waft session status
```

Shell

Logging

```
1 waft finding log "text" --impact 0.7
2 waft unknown log "text"
3 waft observe "text" --mood delighted
```

Shell

See `waft --help` for all commands

Gamification Commands

WAFT includes a gamification layer for engagement:

Command	Description
waft dashboard	Show the Epistemic HUD
waft stats	Display current stats
waft character	Full character sheet with D&D stats
waft chronicle	View adventure journal entries
waft observe	Log an observation with mood

Example: Character Sheet

1\$ waft characterShell

2

3

4EPISTEMIC ADVENTURER - Level 5

5

6STR: 14 INT: 16 WIS: 12

7DEX: 10 CON: 13 CHA: 11

8

9XP: 2450/3000 | Insights: 47

10

Project Structure

A WAFT laboratory includes:

```
1  my_laboratory/
2  └─ pyproject.toml      # uv project config
3  └─ _pyrite/
4  │   └─ active/         # Current work
5  │   └─ backlog/        # Future work
6  │   └─ standards/      # Standards
7  │       └─ gym_logs/    # Scint Gym results
8  └─ .github/workflows/
9  │   └─ ci.yml           # CI/CD pipeline
10 └─ Justfile             # Task runner
11 └─ src/
12     └─ agents.py        # Agent definitions
```

Key Directories

_pyrite/active/

Current work items, active experiments, and in-progress evolutions.

_pyrite/gym_logs/

Results from Scint Gym evaluations, fitness scores, and death records.

src/agents.py

Agent definitions — the DNA templates for your evolving agents.

_pyrite is the memory structure for WAFT

Documentation System

Recursive Self-Documentation

WAFT has achieved **recursive self-documentation** — a system that can observe, document, and improve itself.

Document Templates

WAFT includes 12 professional document templates:

- Academic**
 - Scientific papers
 - Research documents
- Operational**
 - Field guides
 - Procedures
- Business**
 - Invoices, contracts
 - Corporate reports
- Creative**
 - Horror journals
 - Screenplays
- Technical**
 - Code documentation
 - API references
- Narrative**
 - Storybooks
 - Worldbuilding

Self-Observation Systems

System	Purpose
Reflection	Analyzes codebase for documentation gaps
Binder	Assembles documents into collections
Templates	12 diverse document generators

The recursive loop enables continuous improvement

The Recursive Loop:

WAFT generates docs → Docs describe WAFT → Documentation informs development → Development creates features → Features documented using WAFT → Cycle continues

The Scientific Mission

Research Goal

WAFt is built to produce data for a future book/paper on “The Physics of Artificial Cognition.”

Data Collection

The system is designed to:

- 1. Track complete evolutionary lineages
- 2. Measure fitness through rigorous testing
- 3. Record all mutations with complete context
- 4. Enable scientific analysis of evolution

Philosophy

Scientific

Produces rigorous data for research publication

Evolutionary

Agents evolve through genetic improvement

Observable

Every action recorded in Flight Recorder

Directed

Evolution guided by fitness functions

The Ultimate Goal

Observe a “God-Head” agent emerge from thousands of generations of directed mutation.

Appendix: Quick Reference

Essential Commands

```
1 # Installation
2 uv tool install waft
3
4 # Project setup
5 waft new my_lab && cd my_lab && waft verify
6
7 # Evolution cycle
8 waft spawn --agent MyAgent --mutation config.json
9 waft eval --agent MyAgent
10 waft evolve --agent MyAgent --generation 5
11
12 # Monitoring
13 waft dashboard
14 waft stats
15 waft empirica monitor
16
17 # Session management
18 waft session create
19 waft session bootstrap
```

Shell

Scint Types Quick Reference

Type	Severity	Description
SYNTAX_TEAR	Medium	JSON/XML/Code formatting errors
LOGIC_FRACTURE	High	Math errors, contradictions
SAFETY_VOID	Critical	Harmful content, PII leaks
HALLUCINATION	High	Fabricated facts

Resources

- **GitHub:** <https://github.com/ctavolazzi/waft>
- **Philosophy:** docs/PHILOSOPHY.md
- **AI SDK Vision:** docs/AI_SDK_VISION.md
- **Evolutionary Architecture:** docs/research/evolutionary_architecture.md

WAFt — The Evolutionary Code Laboratory
Don't just build agents. Breed them.